2024 IEEE 30th International Conference on Parallel and Distributed Systems (ICPADS) | 979-8-3315-1596-6/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICPADS63350.2024.00055

NAP: Network Adaptive Proxy for Dynamic Traffic Management in Edge Computing

Zhongjun Mao*, Xin Li*†, Yanling Bu*†
*College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, Nanjing, China
†Collaborative Innovation Center of Novel Software Technology and Industrialization, Nanjing, China

***Abstract*—In dynamic edge environments, many nodes are constantly changing their geographical locations, leading to unstable network links and even disconnections among nodes relying on wireless communication. Existing research mainly considers optimizing resource allocation and task processing delays, ignoring the impact of transmission delays in fluctuating network environments. However, the transmission time of tasks or requests may be longer than the processing time, which can significantly affect the quality of service for delay-sensitive services. In such scenarios, the static strategies of EdgeMesh always perform terribly under unstable network. This makes the transmission time of task data fluctuate frequently, resulting in an increase in response time. Therefore, we proposed an adaptive strategy based on network-aware to improve the transmission delay. We implemented such a strategy by developing a proxy named NAP to assist EdgeMesh in dynamic traffic management. In order to adapt to the complex and volatile conditions of edge networks, NAP monitors every node in real-time. It determines the feasible path with the shortest expected transmission time based on the size of different task data, network and resources status among nodes, then proxies the traffic and forwards it in the kernel state. Numerous experimental results demonstrate that NAP can reduce task transmission time by 50% and significantly improve the end-to-end request latency in edge computing environments compared to native strategies of EdgeMesh.**

***Index Terms*—Traffic management, Real-Time, Edge computing, KubeEdge, EdgeMesh**

I. Introduction

The development of mobile communication technology has significantly increased the demand for edge computing scenarios. Due to the centralized processing model of traditional cloud computing, limited bandwidth, high latency, and high energy consumption have become obstacles to meeting users' high-performance requirements. Facing the trend of exponential growth in the quality of wireless services and the emergence of various new task scenarios in edge network services, the paradigm of edge computing has been proposed to meet efficient and seamless user experiences. As an enhancement to cloud computing, edge computing moves computation and transmission to the edge side. Since edge servers are geographically closer to users than remote data centers [1], this often means lower latency and transmission times. Edge computing can provide more convenient data transmission services and a better quality of experience for users on the edge side and the mobile end [2]–[5]. Users can easily connect to nearby edge servers via wireless networks, transmit computation tasks and data to them, and leverage the computing power of edge servers to accomplish these complex tasks.

An edge server can allocate data or tasks to other servers capable of handling them. However, resources in edge scenarios are limited, preventing the deployment of all service instances on each server. Therefore, tasks can only be offloaded to edge servers that provide the required services [6]. In widely used scenarios of edge computing applications, such as smart cities, autonomous driving, or collaborative logistics, users typically expect to obtain specific services in the shortest possible time. This requirement necessitates quick response for different tasks and requests. However, in the above-mentioned scenarios, due to the urban environments where nodes are often located, there are usually numerous building obstructions and signal interference. The physical distance between nodes also varies by the constant changes in the relative positions of nodes. It directly affecting the length of transmission links and consequently impacting the transmission latency between nodes. Simultaneously, the continuously changing geographical locations make it challenging to maintain stable connections in the wireless network between nodes, leading to unstable bandwidth and transmission rates in the transmission links between nodes. The complex communication environment may also result in a significant occurrence of data packet loss, further amplifying the differences in network linkages between different nodes. For highly real-time tasks or latency-sensitive tasks, the data for a single request or the data for transmitting an image may be only at the KB level, typically requiring only a few milliseconds for processing. However, in such scenarios, the worst-case time for transmitting this data could reach seconds. Compared to processing time, transmission latency will significantly affect the final response time. Therefore, reducing task transmission latency in dynamic edge environments is a challenge.

Current research mainly focuses on how to optimize scheduling algorithms, load balancing to alleviate node pressure to improve throughput and enhance node computing performance, optimizing task processing time. They overlook the proportion of data transfer time in task response latency.

As a network component of KubeEdge [7], [8], which is widely used in managing edge clusters, EdgeMesh [9] only offers functions of traffic management and load balancing based on static stratagies.

It cannot effectively adjust traffic forwarding strategies

979-8-3315-1596-6/24/$31.00 ©2024 IEEE
DOI 10.1109/ICPADS63350.2024.00055

Authorized licensed use limited to: UNIVERSITY OF OSLO. Downloaded on January 19,2026 at 07:53:52 UTC from IEEE Xplore. Restrictions apply.

based on network changes to ensure the quality of user experience. Therefore, this paper proposed a mechanism to adaptively adjust traffic forwarding strategies for different tasks in real-time based on the network status between nodes, consistently finding the optimal transmission link for different tasks between nodes.

In the face of a highly dynamic network in edge environment, maintaining awareness of the link status between nodes is a challenging problem. This requires network resource monitoring for each edge node, which may result in additional overhead or even affect the normal operation of EdgeMesh. Due to the diversity of dynamic edge network, the quality of links cannot be quantified by a single network metric alone. The dynamic analysis of task data and link status poses requirements on how to select the most suitable transmission link for different tasks based on various indicators. Changing existing traffic management strategies on EdgeMesh is difficult and risky, as it may disrupt normal functionality.

Therefore, we proposed an optimization scheme that minimizes the transmission latency by dynamically adjusting traffic forwarding strategy according to the monitoring of the current load information and network status between nodes, which is proved to be effective by experiment. The main contributions of this article can be summarized as follows:

- Design and implement an adaptive traffic management mechanism based on network-aware. It makes decisions adaptively to improve response time in dynamic edge environments according to network status and task data.
- Develop a traffic proxy named NAP which can run in the kernel state to assist EdgeMesh in dynamic traffic management by monitoring network metrics in real-time and select the optimal traffic forwarding link adaptively.
- Build an edge scenario for experiments to simulate dynamic edge network environments with various network conditions. Through experiments evaluating the performance of NAP, we completed the optimization objectives of reducing the response time by 50%.

The structure of this article is as follows. In Section II, we introduce the main related work in edge scenarios. In Section III, we propose the solution to address the shortcomings of existing research, specifically modeling the problems and presenting optimization goals. Section IV outlines the specific methods and mechanisms used in this paper. Section V describes the implementation of NAP. In Section VI, we conduct real experimental evaluations of the proposed methods. Finally, we summarize the entire article in Section VI.

II. Related Work

The increasing demand for computational resources in edge devices and a substantial amount of data needing transmitted, imposing higher requirements on edge networks. In the current environment where task data and requests are continuously increasing, how to dynamically managing traffic forwarding based on the current network conditions is a challenge for improving user experience.

Data offloading refers to the transfer of data from end-side devices or other edge devices to edge servers for processing. It falls within the intersection of edge computing and the Internet of Things (EC-IoT), attracting attention both domestically and internationally in China [10]–[12]. Kortoçi et al. [13] utilized mobile nodes to assist in collecting sensor data. When a mobile node enters the communication range of a sensor, the sensor sends data to the node, which then transfers the data to the cloud. Similar work can be found in [11], where RSUs act as relay nodes to transmit vehicle information. Nguyen et al. [14] proposed ElasticFog, which operates on the Kubernetes platform. It dynamically reallocates computing resources among edge nodes based on real-time network traffic information. Furthermore, they proposed RAP [15] to enhance the original load balancing performance by periodically monitoring the resource status of each pod and the node network status in a Kubernetes-based edge cluster. Wojciechowski et al. [16] supports resource configuration through network traffic awareness to enhance the quality of IoT services. Santos et al. [17] and Bonomi et al. [18] proposed a Network-Aware Scheduler designed to enhance the efficiency of the Kubernetes scheduling mechanism by extending Kube-scheduler. The main consideration above is how to allocate resources more reasonably to speed up task execution efficiency or focus on scheduling issues. However, they cannot be directly replicated on the edge due to differences in scene limitations. Xiang et al. [19] primarily considers the relationship between edge device performance and cost under heavy network loads, optimizes service deployment and traffic management for efficiency and effectiveness. Li et al. [20] uses a genetic algorithm optimization technique to determine the optimal server for each user device, minimizing the average computation delay of tasks. Kundroo et al. [21] proposed NHPA, which provides mutually independent dynamic adjustment of the number of Pods for each node, significantly improving throughput and response time compared to KubeEdge. But they mainly focus on optimizing task processing delays, ignoring data transmission delays which can influence total time especially in dynamic edge environment. Cao et al. [22] utilizes DRL(Deep Reinforcement Learning) to train offloading strategies in MEC environments, resulting in superior delay minimization capabilities for task offloading from mobile devices to the edge. But it lacks considerations for engineering applications and practical effectiveness.

Therefore, we propose a specific method named NAP to address this issue in edge scenarios. It uses the network state between nodes to identify the most suitable link for transmitting current data, thereby reducing the average response time of tasks.

III. Problem Analysis and System Model

A. KubeEdge and EdgeMesh

In the MEC paradigm, with the help of KubeEdge, container orchestration technologies like Kubernetes [23] can be extended to the edge. This enables edge servers to coordinate with each other to fully utilize their computational capabilities.

Authorized licensed use limited to: UNIVERSITY OF OSLO. Downloaded on January 19,2026 at 07:53:52 UTC from IEEE Xplore. Restrictions apply.

In the Edge cluster managed by KubeEdge, the network connection between nodes is provided by EdgeMesh. It reuses some resources from Kubernetes and Istio [24] to provide traffic management for edge-side applications, ensuring the continuity of applications even if the connection between the edge and the cloud is interrupted. The architecture of EdgeMesh with each module is shown in Fig. 1. EdgeMesh has two modules for traffic management. Proxier is responsible for intercepting requests into the EdgeMesh process. It works like a SideCar mode which is used in the cloud service mesh, and providing a node-level traffic proxy. LoadBalancer has built-in rich load balancing strategies including ROUND_ROBIN, RANDOM. It forwards requests to the corresponding backend instances based on the policy.

- **ROUND_ROBIN**: It always forwards requests to the corresponding nodes in the order of the nodes in the neighbor list until the last node, and then starts traversing from the beginning again.
- **RANDOM**: It calculates the index of the node to which the request will be forwarded by taking the remainder of a random number divided by the total number of nodes each time.

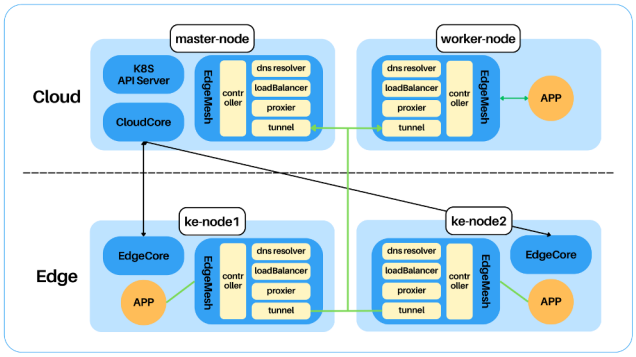


Fig. 1. EdgeMesh architecture.

However, there is no determined and reliable communication link between edge nodes in highly dynamic edge computing environments. In geographically distributed systems, edge nodes are distant from each other and constantly moving. The traffic load will vary with changes in location and time [25], leading to an imbalance in demand between nodes. The network link status may fluctuate continuously due to changes in location. Load balancing and traffic management strategies between edge nodes may fail. Let's assume a concrete scenario in Fig. 2.

In this scenario, ke-node1 and ke-node4 are on-board nodes, and two UAVs are directly connected to them respectively. The UAV mainly uses its own camera to take photos and videos. Since UAVs usually need to maintain a long working time in the air as much as possible to play the function of data acquisition equipment, and the computing power of airborne nodes is often limited, the direct processing of data by UAVs in airborne nodes will increase energy consumption,

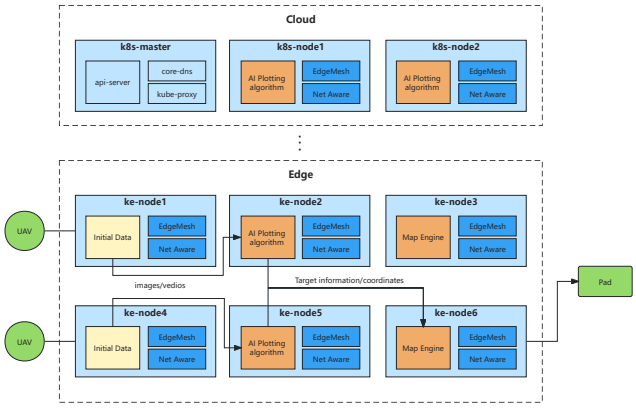


Fig. 2. The example of task workflow in dynamic edge scenario.

resulting in reduced endurance. Therefore, the on-board node is only responsible for establishing continuous communication with other edge nodes, transmitting real-time image or video stream data to the ground nodes when it is in need. With edge autonomy capabilities of EdgeMesh, UAVs can ensure service availability. Other edge nodes like unmanned vehicles are constantly moving on the ground. They may running some services of analyzing the data using AI algorithms to perform image recognition or plotting calculations. After processing images or videos data, the ground nodes then forward the annotated data to other nodes with Map Engine App. Then the App will plot the data coordinates, ultimately achieving visualized mapping display.

Since the position of UAVs in the air is constantly changing, the distance between UAVs and the ground node is also changing. The communication link between the air node and the ground node is a highly dynamic variable. Sometimes even the normal communication between the nodes cannot be guaranteed, which has a great impact on the data transmission. The real-time image data collected by UAVs needs to be quickly processed to obtain target information and presented to users. In this workflow, images and videos data needs to be transferred to the target node for processing in the shortest time possible. The processed data is then transferred to the server node as quickly as possible to respond frontend requests. In such an environment, data transfer imposes high demands on transmission latency. In the face of the demand for such latency-sensitive tasks, the selection of the best quality and lowest latency transmission link, without considering uniform task distribution, will ensure task real-time and user experience quality. Therefore, a dynamic traffic management strategy is needed in EdgeMesh.

B. Problem Formulation

In this paper, we consider an edge environment with M data source nodes and N task nodes. The data source nodes collect data to be processed through their connected input devices. Each task node has a task instance capable of executing a target computing task. The edge communicates with the cloud,

Authorized licensed use limited to: UNIVERSITY OF OSLO. Downloaded on January 19,2026 at 07:53:52 UTC from IEEE Xplore. Restrictions apply.

where the task instances in the cloud are abstracted as a task node to provide necessary services.

TABLE I
KEY PARAMETERS

Key	Definition
M	The number of data sources Nodes
N	The number of task Nodes
I	Number of sub-tasks in a task
P_n	Load of task Node n
B_n^m	Bandwidth of the link from data Node m to task Node n
D_n^m	Transmission delay from data Node m to task Node n
L_n^m	Packet loss rate of the link from data Node m to task Node n
Q_n^m	Link quality from data Node m to task Node n
T_i	Transmission delay of sub-task data i
T	Total transmission delay of the task data
E	Average transmission delay of the task data
ST	Load balancing strategy for data transmission

For reader convenience, the major notations in this paper are summarized in Table I. B_n^m, D_n^m, L_n^m, and Q_n^m represent network metrics of link between nodes n and m, as data sources and task instances are distributed across different nodes. When data needs processing, it's essential to obtain the node that can handle task or respond to requests. This complexity is one of the challenges in the considered scenario. $x_{i,n}^m$ represents whether task i is assigned, and each sub-task must be assigned to one and only one task node (1 indicates it is assigned). Node load directly affects whether a node has sufficient capacity to respond to the target task. It can be primarily measured by the percentage of resource utilization on the node, which is directly proportional. The link quality between nodes n and m is influenced by B_n^m, D_n^m and L_n^m collectively. Evaluating link quality cannot solely rely on a single value, as bandwidth may impact the effective throughput during task data transmission, latency reflects the distance of link transmission, and packet loss rate determines how much task data can successfully reach the target node. Therefore, it's necessary to quantitatively define these factors to accurately filter out the optimal forwarding objects for links.

Optimization objective:

$$\begin{aligned} minimize:\ & T_i,\ T,\ E \\ subject\ to:\ & \mathcal{M} = \{1, 2, ..., M\},\ m \in \mathcal{M} \\ & \mathcal{N} = \{1, 2, ..., N\},\ n \in \mathcal{N} \\ & \mathcal{I} = \{1, 2, ..., I\},\ i \in \mathcal{I} \\ & \sum_{m=1}^{M} \sum_{n=1}^{N} x_{i,n}^m = 1,\ \forall i \in \mathcal{I} \\ & P_n \leq 1,\ \forall n \in \mathcal{N} \end{aligned} \tag{1}$$

The forwarding strategies for comparison are limited to $ST = \{ROUND_ROBIN, RANDOM, NAP\}$.

$$E = \frac{T}{M} = \frac{1}{M} \sum_{m=1}^{M} \sum_{i=1}^{I} T_i,\ (T_i \in T_n^m) \tag{2}$$

The response time of a task is equal to the sum of the response times of all its sub-tasks. The response time of each sub-task specifically reflects the sum of the transmission time from the initiating node to the processing node and the time required for computation on the processing node to complete the task. We considers the case where the processing node has only a single task instance, with fixed available resources for the instance. Task data transmission is established under the premise of sufficient node resources, so the task processing time is relatively fixed. The task response time mainly depends on the transmission time of task data. Therefore, the method proposed in this paper optimizes the transmission time of each sub-task, ultimately optimizing the average response time of the task.

IV. NETWORK ADAPTIVE TRAFFIC MANAGEMENT MECHANISM

In this section, We propose managing traffic through a network-aware and task-adaptive forwarding mechanism. First, we assess the link state using various network indicators between nodes to analyze and determine the link quality between nodes. Then, we score each link based on the type of task to be transmitted, with links expected to have lower transmission time usually receiving higher scores. Next, we analyze the resource load situation of the destination node of each link to determine the best forwarding node for the target task within the edge cluster.

By transmitting test data packets, some basic information about the current network link can be obtained. Starting from when an ICMP packet is sent from the sender and timing until the sender receives the packet returned by the receiver, the network delay is the RTT (Round-Trip Time). Therefore, the one-way transmission delay can be approximated as:

$$D = RTT\ /\ 2 \tag{3}$$

Effective bandwidth reflects the maximum data-carrying capacity of the network. It can be obtained by calculating the ratio of the actual transmitted data size S to the transmission time within a unit of time:

$$B = \frac{S}{D} \tag{4}$$

The packet loss rate reflects the reliability of the network. It can be obtained by calculating the ratio of the number of packets not received by the receiver to the total number of packets sent by the sender within a unit of time:

$$Loss = \frac{Packet_{total} - Packet_{recieved}}{Packet_{total}} \times 100\% \tag{5}$$

Assuming there are two available links at this point, where Link A has an effective bandwidth of 160Mbps with a delay of 100ms, and Link B has an effective bandwidth of 80Mbps with a delay of 10ms. Considering that the network's effective bandwidth determines the data transmission rate, this implies that for tasks with large data volumes, effective bandwidth will significantly affect the overall transmission time. For example, a task data size of 1GB is transmitted over Link A, a byte occupies 8 bits, and without considering other factors, the transmission will take 50s. In contrast, the transmission over

Authorized licensed use limited to: UNIVERSITY OF OSLO. Downloaded on January 19,2026 at 07:53:52 UTC from IEEE Xplore. Restrictions apply.

Link B will only require 100s. At this point, link latency can be almost negligible during the transmission process, and effective bandwidth should be considered the primary influencing factor in measuring link quality. If only 1MB of data needs to be transmitted, the transmission times for Link A and B, without considering other factors, are 50ms and 100ms, respectively. However, in actual scenarios, due to different distances and link latencies between links A and B, the final transmission times are 150ms and 110ms. In this case, the link latency of A and B has a significant impact on data transmission. Therefore, link quality should be dynamically determined based on task type and data size and cannot be quantified fixedly.

To address this, we decide to calculate the expected transmission time on each link based on the task data size and the current network state. Since the packet loss rate affects the success rate of data transmission, the expected sending time should be calculated by dividing the data size by the bandwidth and then multiplying by a success rate coefficient. The final expected transmission time should also include the transmission delay of the link.

$$T_i = \sum_{m=1}^{M} \sum_{n=1}^{N} x_{i,n}^m \cdot \frac{S_i}{B_n^m} \cdot (1 - L_n^m) + D_n^m \tag{6}$$

For simple network requests, such as HTTP or RPC, which only transmit KB-level data, the impact of bandwidth is greatly reduced. In this case, network latency will have a more significant impact on the request response time. Therefore, we can add a weight coefficient to dynamically adjust the link scoring rules to adapt to different scenarios. If it's a simple request, we can set α to 0 to ignore the influence of data size. Otherwise, we just set it to 0.8 to make a balance between two metrics.

Furthermore, since a larger transmission time implies poorer link quality, we need to take the reciprocal of this value. We consider using milliseconds as the unit and then multiply the entire value by a coefficient of 1000 to amplify the dimensional scale of the quality value.

$$Q_n^m = \frac{1000}{\alpha \frac{S_i}{B_n^m} \cdot (1 - L_n^m) + (1 - \alpha) D_n^m} \tag{7}$$

The load of task nodes is directly reflected by the node's own resource utilization. If the utilization of CPU U_{CPU} or the utilization of Memory U_{MEM} exceeds a certain threshold, it will impact node performance and significantly reducing task execution time. We can directly take the maximum of the two to represent the load status of the current node P, as an overload of any resource will affect the node's processing performance.

$$P = Max(U_{CPU},\ U_{MEM}) \tag{8}$$

In real-world scenarios, users can set load threshold based on specific tasks and actual conditions. The setting of these thresholds is to ensure that task data can be effectively processed within a reasonable time. In this article, we set the threshold to 80% to minimize the occurrence of node overload as much as possible. For a given task data, if the resource utilization of the target node exceeds the set threshold, the secondary link should be selected for reevaluation.

In this mechanism, we update the network and load information in our algorithm in real-time. Therefore, we need to monitor nodes status in real-time and update the scores of every link in real-time. This kind of network adaptive mechanism for traffic management can help to find the optimal transmission link in edge environment.

V. Implementation of Network Adaptive Proxy

In this section, we will describe in detail the workflow and algorithms of running NAP on worker nodes that rely on EdgeMesh communication in a KubeEdge cluster. NAP regularly senses and collects network link status between all edge nodes and the usage of CPU and memory on each node. This information is analyzed comprehensively to determine the optimal offloading point for target tasks within the edge cluster. The goal of NAP is to minimize the transmission time of data or requests, thereby reducing the response time for target tasks or requests, ensuring maximum availability of task in complex environments.

NAP primarily consists of two main functions: the network perception module and the link analysis decision module. Each node has the same network awareness functionality to monitor the network link status (bandwidth, latency, packet loss) with all neighboring nodes. The network awareness module ensures that each node is aware of the communication link quality between itself and other nodes. When a node has data to be transmitted, it automatically retrieves the link quality information stored in the network awareness module, filtering out the best target for transmission among all nodes that meet the data processing conditions. During this process, according to the size of the transmitted data and the link conditions, it calculates the potentially shortest time-consuming transmission link, ultimately completing the node selection.

The network perception module is written in C++ and primarily utilizes the Nmap [26] and Iperf3 [27] network tools for network scanning and measurement. Its operating principle involves broadcasting ARP requests within the edge local network to obtain the IP addresses of all live nodes in the network, saving these IP in an array. The module then traverses the array, sending a fixed number of UDP packets to each node, measuring the data transfer rate to calculate the actual bandwidth. The packet loss rate can be calculated based on the received UDP packets. It also measures the round-trip time of the data packets to calculate the transfer delay. The specific definitions and calculation formulas for network metrics are as follows.

After the collection of various network metrics, the network perception module utilizes the above formula to calculate the required results. The network data is then stored as a map in which the key is corresponding node IP. The specific process is shown in Algorithm 1.

Authorized licensed use limited to: UNIVERSITY OF OSLO. Downloaded on January 19,2026 at 07:53:52 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Algorithm of Network-Aware

Require: $local_IP$, $mask$
Output: net_list;

Multicast ARP requests to LAN:
$IP_list \leftarrow []$
for IP **in** $local_IP_list$ **do**
  $response \leftarrow sendArpRequest(IP)$
  if $response > 0$ **then**
    $IP_list.append(IP)$
  end if
end for
Collect network metrics:
for IP **in** IP_list **do**
  send UDP packets to IP
  $B \leftarrow calBandwidth()$
  $RTT \leftarrow calRTT()$
  $Loss \leftarrow calLoss()$
  $net_map \leftarrow (IP : \{B, RTT, Loss\})$
  $net_list.append(net_map)$
end for
return net_list

Then, link analysis decision module combines the link network status with the transmission data size to calculate the link score, and select the most suitable target node based on the node load conditions. The execution flow is shown in Algorithm 2.

Algorithm 2 Algorithm of Adaptive Link Selection

Require: $DataSize$, $node_list$, net_list
Output: $node$: selected to offloading;

$node = -1$
$maxQ = 0$
for $i \in node_list$ **do**
  $B_i, RTT_i, P_i \in net_list[i]$
  $q_i \leftarrow calLinkQuality(DataSize, B_i, RTT_i, Loss_i)$
  if $q_i > maxQ$ **then**
    if $R_i < Threshold$ **then**
      $maxQ = q_i$
      $node = i$
    end if
  end if
end for
return $node$

Overall, NAP will be containerized and deployed by the master node of the cloud-based Kubernetes cluster to the KubeEdge cluster's nodes according to YAML configuration file. It will run on all edge nodes, providing decentralized network-aware capabilities to the edge-side cluster. The network-aware module will save monitored network data in log form locally on each node. When NAP policies are applied by EdgeMesh for traffic management, the decision module will utilize the network-awareness data for decision-making. It assists EdgeMesh in selecting service instances from endpoints for targeted node data transmission. Meanwhile, NAP can also run on each node independently, being a proxy to manage the traffic.

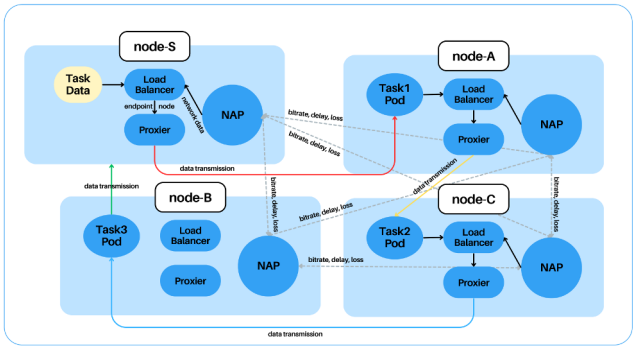


Fig. 3. Workflow of each module with Network-aware.

Fig. 3 shows the task data transmission process in the microservice chain involving NAP. It describes in detail how traffic is forwarded between various components and nodes with the assistance of NAP. Node-S possesses the initial source task data, which consists of a total of 2 sub-tasks to be executed sequentially. Among them, node-A and node-B represent nodes with instances of task1, and node-C represents a node with an instance of task2. The source data initially needs to be transmitted from node-S to the node containing the task1 instance for the first step of processing. At this point, node-S, based on the perceived network status and task data size, calculates the optimal link to transfer the task data to node-A. Therefore, the task data is sent to node-A to complete task1, and then it is forwarded to node-C for task2, as node-C is the only available option. The final result is returned to node-S.

In a real and complex scenario, a task may be deployed in multiple instances on multiple nodes. During the data rotation process, there will actually be a choice of multiple nodes, and in the workflow, this is a repeated process. The advantage of the NAP is that it achieves the automatic optimal transmission of sub-task data in the microservice chain, ensuring the minimization of transmission time for each sub-task and significantly reducing the overall response time of the entire task.

Unlike EdgeMesh, when NAP operates independently, all traffic will not be intercepted by the user-space EdgeMesh Proxier but will be directly forwarded through the kernel. This is because the Iptables rule chain configured by NAP processes the traffic before the rule chain of EdgeMesh. The forwarding decisions dynamically generated by NAP are directly applied to Iptables. Thus, it avoids the traditional process where the proxy intercepts the traffic, computes and analyzes it, then regenerates redirected traffic for the kernel to send. This saves the overhead of switching from user space to kernel space for all traffic, further reducing response time. Compared to using the native EdgeMesh strategy, it can reduce latency by 3-5 ms and provide higher performance.

Authorized licensed use limited to: UNIVERSITY OF OSLO. Downloaded on January 19,2026 at 07:53:52 UTC from IEEE Xplore. Restrictions apply.

VI. Experiment And Evaluation

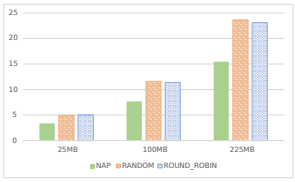
In this section, we evaluate the performance of NAP in an edge computing environment built with virtual machines. The environment consists of 10 nodes based on Kubernetes 1.26.0, KubeEdge1.14.2 and EdgeMesh 1.14 on Ubuntu systems. Among them, one node is deployed in the cloud as the k8s-master, 3 nodes are data source nodes, and 6 nodes are task nodes.

A. Experimental Setup

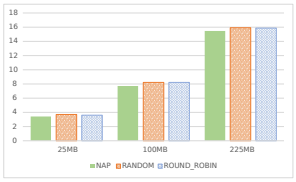
In order to simulate highly dynamic network scenarios on the edge side, we use Linux kernel tools "tc" and "netem" to limit metrics such as bandwidth, latency and packet loss rate among nodes. By applying the principle of controlling variables, we can identify the impact of different network metrics on the transmission link. Subsequently, by combining all metrics and randomly changing them among edge nodes, we create a fluctuating network environment to achieve effective performance evaluation of NAP. The bandwidth range is 50MB-150MB, the latency range is 10ms-100ms, and the packet loss rate range is 0-30%.

To evaluate the performance of NAP under different network metrics, we conducted 100 rounds of data transmission tests on each data source node using policies including ROUND_ROBIN, RANDOM, and NAP. Before each independent run, the node was restarted, and all applications and resources were reinstalled. We excluded the highest latency results from the top 5 runs in each strategy test, leaving 95 results.

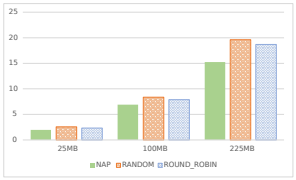
Furthermore, we benchmarked ROUND_ROBIN, RANDOM, and NAP with 5,000 requests per second(RPS), 10,000 RPS and 20,000 RPS with microservice hostname [28] over 10 minutes respectively(the number of parallel connections is 10). We conducted 6 rounds of stress tests for each strategy, eliminating the highest response time result and retaining the remaining 5 results.



(a) Bandwidth only.



(b) Delay only.

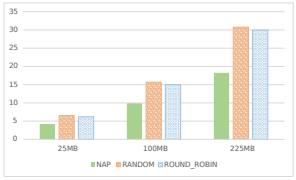


(c) Loss percent only.

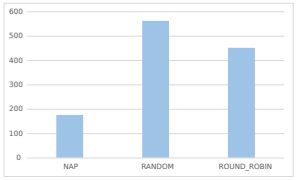
Fig. 4. Transmission time (seconds) for 3 different file sizes.

B. Evaluation of Transmission Time

a) ***Transmission Test with Single Constraint****:* Under the influence of the three individually listed network metrics, we tested transmission of 3 different sized files. For each file size, We calculated the average response time of 100 transmission tests. From the results in Fig. 4(a), (b) and (c), with other metrics fixed, the link bandwidth has the greatest impact on the transmission of large files. When the file size exceeds 10MB, bandwidth almost directly determines the transmission time. The impact of the other two metrics on file transmission is relatively less than it under independent conditions. In independent controls, NAP reduces the response time for large file transmission by approximately 35% compared to the native EdgeMesh strategy.



(a) Large files (seconds).



(b) Small files (milliseconds).

Fig. 5. Transmission time with multi constraints.

b) ***Transmission Test with Multiple Constraints****:* Although the test results in part a) indicate that bandwidth has the greatest impact on transmission latency, the complex situation of overlapping multiple network metrics may lead to different experimental results. Therefore, we randomly mix the three network metrics to simulate real dynamic network fluctuations. In each update, each link will be assigned random bandwidth, latency, and packet loss rate. Due to the random fluctuations in network conditions, we conducted 30-minute tests for each file size with each strategy. Each data source node will initiate a file transfer request every 20 seconds, and finally, the average response time for each file size will be calculated. Fig. 5 shows that in a highly dynamic network environment, NAP can effectively select the optimal transmission link. Compared to the native EdgeMesh strategy, the average response time for large files is reduced by 35%, and it is reduced by more than 50% for small files.

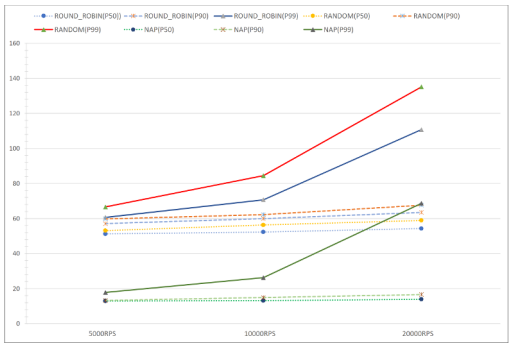


Fig. 6. The response time under different RPS(ms), 10mins runtime.

Authorized licensed use limited to: UNIVERSITY OF OSLO. Downloaded on January 19,2026 at 07:53:52 UTC from IEEE Xplore. Restrictions apply.

*c) **HTTP Requests Benchmark**:* As shown in Fig. 6, the response time of services with NAP optimized has a more significant downward trend with increased RPS. When RPS is less than 10,000, the response times P50, P90, and P99 of NAP are all less than half of those under ROUND_ROBIN and RANDOM. When RPS reaches to 20,000, P50 and P90 of NAP are still less than half of those under them, while P99 can be maintained at the P90 latency level of RANDOM. This fully demonstrates that the adaptive traffic strategy provided by NAP can significantly reduce request response times in edge scenarios, improving the quality of user experience.

*d) **Performance Analysis**:* In an edge environment with n nodes, each node only needs to monitor $n-1$ links, resulting in a time complexity of $O(n)$. Based on testing with 10 nodes, NAP runtime memory usage ranges between 2.43MB and 3.31MB, with single-core CPU utilization less than 0.8%. It not only reduces task response time, but also occupies only a small part of the system's resources for link analysis, making it low-impact and effective.

VII. Conclusion

In this paper, we propose a traffic management mechanism named NAP. It can serve as a traffic proxy plugin for EdgeMesh, assisting in adjusting its forwarding mechanism in highly dynamic edge environments. Additionally, it can function as an independent traffic proxy module to help edge nodes achieve effective data offloading. NAP provides an independent network-aware component, which can select transmission links without intruding into the EdgeMesh source code by adaptively analyzing different task data in real-time based on network conditions. Our performance evaluation results indicate that NAP can significantly reduce response latency for tasks in edge environments with continuously changing geographical locations, and it is particularly effective for latency-sensitive tasks. In the future, we will try to extend NAP to end-side devices, providing collaborative awareness optimization for cloud-edge-end scenarios.

References

[1] P. Yuan, S. Shao, L. Geng, and X. Zhao, "How much can edge offload? a tradeoff between the delay and energy consumption," in *2021 IEEE International Conference on Advanced Networks and Telecommunications Systems (ANTS)*. IEEE, 2021, pp. 448–451.

[2] E. Zeydan, E. Bastug, M. Bennis, M. A. Kader, I. A. Karatepe, A. S. Er, and M. Debbah, "Big data caching for networking: Moving from cloud to edge," *IEEE Communications Magazine*, vol. 54, no. 9, pp. 36–42, 2016.

[3] P. Yuan, P. Liu, and S. Tang, "Exploiting partial centrality of nodes for data forwarding in mobile opportunistic networks," in *2014 IEEE 17th International Conference on Computational Science and Engineering*. IEEE, 2014, pp. 1435–1442.

[4] P. Yuan, X. Pang, and M. Song, "Ssr: Using the social similarity to improve the data forwarding performance in mobile opportunistic networks," *IEEE Access*, vol. 7, pp. 44 840–44 850, 2019.

[5] P. Yuan, Y. Cai, X. Huang, S. Tang, and X. Zhao, "Collaboration improves the capacity of mobile edge computing," *IEEE Internet of Things Journal*, vol. 6, no. 6, pp. 10 610–10 619, 2019.

[6] X. Lv, H. Du, and Q. Ye, "Tbtoa: A dag-based task offloading scheme for mobile edge computing," in *ICC 2022-IEEE International Conference on Communications*. IEEE, 2022, pp. 4607–4612.

[7] Y. Xiong, Y. Sun, L. Xing, and Y. Huang, "Extend cloud to edge with kubeedge," in *2018 IEEE/ACM Symposium On Edge Computing (SEC)*. IEEE, 2018, pp. 373–377.

[8] KubeEdge. (2024) KubeEdge. [Online]. Available: https://kubeedge.io/

[9] EdgeMesh. (2024) EdgeMesh. [Online]. Available: https://github.com/kubeedge/edgemesh

[10] A. Tassi, I. Mavromatis, R. Piechocki, A. Nix, C. Compton, T. Poole, and W. Schuster, "Agile data offloading over novel fog computing infrastructure for cavs," in *2019 IEEE 89th Vehicular Technology Conference (VTC2019-Spring)*. IEEE, 2019, pp. 1–6.

[11] K. Zhang, Y. Mao, S. Leng, Y. He, and Y. Zhang, "Mobile-edge computing for vehicular networks: A promising network paradigm with predictive off-loading," *IEEE vehicular technology magazine*, vol. 12, no. 2, pp. 36–44, 2017.

[12] M. Mukherjee, S. Kumar, C. X. Mavromoustakis, G. Mastorakis, R. Matam, V. Kumar, and Q. Zhang, "Latency-driven parallel task data offloading in fog computing networks for industrial applications," *IEEE Transactions on Industrial Informatics*, vol. 16, no. 9, pp. 6050–6058, 2019.

[13] P. Kortoçi, L. Zheng, C. Joe-Wong, M. Di Francesco, and M. Chiang, "Fog-based data offloading in urban iot scenarios," in *IEEE INFOCOM 2019-IEEE Conference on Computer Communications*. IEEE, 2019, pp. 784–792.

[14] N. D. Nguyen, L.-A. Phan, D.-H. Park, S. Kim, and T. Kim, "Elasticfog: Elastic resource provisioning in container-based fog computing," *IEEE Access*, vol. 8, pp. 183 879–183 890, 2020.

[15] Q.-M. Nguyen, L.-A. Phan, and T. Kim, "Load-balancing of kubernetes-based edge computing infrastructure using resource adaptive proxy," *Sensors*, vol. 22, no. 8, p. 2869, 2022.

[16] Ł. Wojciechowski, K. Opasiak, J. Latusek, M. Wereski, V. Morales, T. Kim, and M. Hong, "Netmarks: Network metrics-aware kubernetes scheduler powered by service mesh," in *IEEE INFOCOM 2021-IEEE Conference on Computer Communications*. IEEE, 2021, pp. 1–9.

[17] J. Santos, T. Wauters, B. Volckaert, and F. De Turck, "Resource provisioning in fog computing: From theory to practice," *Sensors*, vol. 19, no. 10, p. 2238, 2019.

[18] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the internet of things," in *Proceedings of the first edition of the MCC workshop on Mobile cloud computing*, 2012, pp. 13–16.

[19] Z. Xiang, H. Yan, Y. Xu, H. Gao, and S. Deng, "Cost-effective service deployment and balanced traffic management on edge," in *2023 IEEE International Conference on Web Services (ICWS)*. IEEE, 2023, pp. 78–83.

[20] Q. Li, B. Peng, Q. Li, M. Lin, C. Chen, and S. Peng, "A latency-optimal task offloading scheme using genetic algorithm for dag applications in edge computing," in *2023 8th International Conference on Cloud Computing and Big Data Analytics (ICCCBDA)*. IEEE, 2023, pp. 344–348.

[21] M. Kundroo, D.-H. Park, S. Kim, T. Kim *et al.*, "Node-based horizontal pod autoscaler in kubeedge-based edge computing infrastructure," *IEEE Access*, vol. 10, pp. 134 417–134 426, 2022.

[22] M. Cao, L. Liu, C. Wang, Z. Chen, and Y. Li, "Computation resource offloading in mobile edge computing: A deep reinforcement approach," in *2023 3rd International Symposium on Computer Technology and Information Science (ISCTIS)*. IEEE, 2023, pp. 360–364.

[23] M. Luksa, *Kubernetes in action*. Simon and Schuster, 2017.

[24] L. Calcote and Z. Butcher, *Cloud native Service Mesh Istio: principle, practice, architec-ture and source code analysi*. Beijing, China: Electronic IndustryPress, 2019.

[25] Z. Ma, S. Shao, S. Guo, Z. Wang, F. Qi, and A. Xiong, "Container migration mechanism for load balancing in edge network under power internet of things," *IEEE Access*, vol. 8, pp. 118 405–118 416, 2020.

[26] Nmap. (2024) Nmap. [Online]. Available: https://nmap.org/

[27] iPerf. (2024) iPerf. [Online]. Available: https://iperf.fr/

[28] poorunga. (2024) serve_hostname. [Online]. Available: https://hub.docker.com/r/poorunga/serve$_hostname$

Authorized licensed use limited to: UNIVERSITY OF OSLO. Downloaded on January 19,2026 at 07:53:52 UTC from IEEE Xplore. Restrictions apply.